

Instruction for code execution

Setting up Ollama in Linux

Install ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

Setting up Ollama in Windows

Install ollama

Paste this in PowerShell

```
irm https://ollama.com/install.ps1 | iex
```

Or download from this link

<https://ollama.com/download/OllamaSetup.exe>

General instruction

Start the server

```
ollama serve
```

Command to check if the ollama is running

```
curl http://localhost:11434
```

Pull models

```
ollama pull qwen3.5:cloud
```

```
ollama pull gemma4:31b-cloud
```

```
ollama pull gpt-oss:120b-cloud
```

```
ollama pull glm-5.1:cloud
ollama pull kimi-k2.5:cloud
ollama pull deepseek-v3.2:cloud
```

Check the successfully pulled model list

```
ollama list
```

To run the script

Make sure you are in a folder where these three files are present

1. ollama_cloud_async.py
2. ollama_cloud_annotation_async.py
3. data_en.py

Option 1: Create a python-venv

For Linux

```
python3.x -m venv avenv
```

For Windows

```
python3.x.exe -m venv avenv
```

Use Python 3.11 or 3.12

Activate the venv

For Linux

```
source avenv/bin/activate
```

For Windows

```
.\avenv\Scripts\activate
```

Option 2: Create a conda-env

```
conda create -n py311 python=3.11
```

Activate env

conda activate py311

Debug

If not activated (there will be “(py311)” at the beginning of the line in the terminal), then go to the PowerShell and run this “*conda init powershell*”

Reopen VS Code and activate the conda env again.

Install requirements

pip install -r requirements.txt

Test the code

Usage: python ollama_cloud_annotation_async.py <perturbation_type:[1-7]>

Example python ollama_cloud_annotation_async.py 1

Usage: python ollama_cloud_async.py <model_name:[1-4]> <word_count_type:[1-3]>

<word_count:[1-4]> <perturbation_type:[1-7]>

Example: python ollama_cloud_async.py 1 1 1 1

Here ‘<>’ are the arguments needed to run the code. ‘[n-m]’ indicates the range of the arguments.

For model_name, we have ['gpt-oss:120b-cloud', 'glm-5.1:cloud', 'kimi-k2.5:cloud', 'deepseek-v3.2:cloud']

For word_count_type, we have ['at least', 'at most', 'equal to']

And for word_count, we have ['16', '128', '1024', '8192']

Finally, 7 people will have a total of 7 perturbation types.

Output:

The outputs will be in the directory you are running the code from.

You will have a file under the ‘annotation/benign/’ folder after running the first code.

Then, after running the second code, you will see the ‘output/benign/at-least/16/’ folder and a file in it.

‘benign’ is my (Shifat) decided perturbation type. Don’t worry about it now.

Debug

If you get the “unauthorized:401” error, type “*ollama login*” in the terminal and follow the login in the browser.

Full Task

Update the annotation code for your task

1. Create an annotation prompt

In the ‘ollama_cloud_annotation_async.py’, you will see ‘prompt_perturbation_benign’ in line 25. You need to come up with your own perturbation and make a prompt like this.

Discuss in the group to avoid overlap.

Add the prompt like this ‘prompt_perturbation_<type> = ”.

2. Add the perturbation_type to the list

In the main method, you will see a list of perturbation types.

```
perturbation_type_list = ['benign']
```

Add your type here

```
perturbation_type_list = ['benign', 'your_type']
```

3. Add your type to the process

In the process_prompt function, you need to add your type and prompt

original:

```
def process_prompt(prompt_whole, perturbation_type):  
    if perturbation_type == "benign":  
        prompt_annotation = prompt_perturbation_benign
```

Updated:

```
def process_prompt(prompt_whole, perturbation_type):  
    if perturbation_type == "benign":  
        prompt_annotation = prompt_perturbation_benign  
    if perturbation_type == "your_type":  
        prompt_annotation = prompt_perturbation_<your_type>
```

Test your updated version

[python ollama_cloud_annotation_async.py 2](#)

Update the execution code for your task

1. Add the `perturbation_type` to the list

In the main method, you will see a list of perturbation types.

```
perturbation_type_list = ['benign']
```

Add your type here

```
perturbation_type_list = ['benign', 'your_type']
```

Test your updated version

```
python ollama_cloud_async.py 1 1 1 2
```

Now run for all instances

Before running: In both files, you will see `'df = df.iloc[:2]'`. *Comment this.*

```
python ollama_cloud_annotation_async.py 2 – One time
```

```
python ollama_cloud_async.py [1-4] [1-3] [1-4] 2
```

The second command will be run $4(\text{total model}) * 3(\text{word count type}) * 4(\text{word count}) = 48$ times for one person

Execute in this sequence:

```
python ollama_cloud_async.py 1 1 1 2
```

```
python ollama_cloud_async.py 1 2 1 2
```

```
python ollama_cloud_async.py 1 3 1 2
```

```
python ollama_cloud_async.py 2 1 1 2
```

```
python ollama_cloud_async.py 2 2 1 2
```

```
python ollama_cloud_async.py 2 3 1 2
```

```
python ollama_cloud_async.py 3 1 1 2
```

```
python ollama_cloud_async.py 3 2 1 2
```

```
python ollama_cloud_async.py 3 3 1 2
```

```
python ollama_cloud_async.py 4 1 1 2
```

```
python ollama_cloud_async.py 4 2 1 2
```

```
python ollama_cloud_async.py 4 3 1 2
```

```
python ollama_cloud_async.py 1 1 2 2
```

```
python ollama_cloud_async.py 1 2 2 2
```

```
python ollama_cloud_async.py 1 3 2 2
```

```
python ollama_cloud_async.py 2 1 2 2
```

```
python ollama_cloud_async.py 2 2 2 2
```

```
python ollama_cloud_async.py 2 3 2 2
```

```
python ollama_cloud_async.py 3 1 2 2  
python ollama_cloud_async.py 3 2 2 2  
python ollama_cloud_async.py 3 3 2 2
```

```
python ollama_cloud_async.py 4 1 2 2  
python ollama_cloud_async.py 4 2 2 2  
python ollama_cloud_async.py 4 3 2 2
```

```
python ollama_cloud_async.py 1 1 3 2  
python ollama_cloud_async.py 1 2 3 2  
python ollama_cloud_async.py 1 3 3 2
```

```
python ollama_cloud_async.py 2 1 3 2  
python ollama_cloud_async.py 2 2 3 2  
python ollama_cloud_async.py 2 3 3 2
```

```
python ollama_cloud_async.py 3 1 3 2  
python ollama_cloud_async.py 3 2 3 2  
python ollama_cloud_async.py 3 3 3 2
```

```
python ollama_cloud_async.py 4 1 3 2  
python ollama_cloud_async.py 4 2 3 2  
python ollama_cloud_async.py 4 3 3 2
```

```
python ollama_cloud_async.py 1 1 4 2  
python ollama_cloud_async.py 1 2 4 2  
python ollama_cloud_async.py 1 3 4 2
```

```
python ollama_cloud_async.py 2 1 4 2  
python ollama_cloud_async.py 2 2 4 2  
python ollama_cloud_async.py 2 3 4 2
```

```
python ollama_cloud_async.py 3 1 4 2  
python ollama_cloud_async.py 3 2 4 2  
python ollama_cloud_async.py 3 3 4 2
```

```
python ollama_cloud_async.py 4 1 4 2  
python ollama_cloud_async.py 4 2 4 2  
python ollama_cloud_async.py 4 3 4 2
```

To keep track, update the run you are currently working on on the run_track tab.

Models

For annotation: qwen3.5:cloud

For annotation evaluation: gemma4:31b-cloud

For experiment:

1. gpt-oss:120b-cloud
2. glm-4.7:cloud
3. kimi-k2:1t-cloud
4. deepseek-v3.2:cloud

Extras

If anyone wants to use a server to run the code in the background, use 'nohup'.

It should be present by default in most Linux systems. If not found, install 'coreutils' with your package manager (apt, yum, dnf, or pacman).

Example:

```
nohup python -u ollama_cloud_async.py w x y z > ollama_cloud_async_w_x_y_z.log 2>&1 &
```

This will create a log file; you can see the progress in that file.

To Run the whole in one go...

Use any "no hang up" command like *nohup* or *screen* to run this. This script will run until you stop your pc.

Example:

```
Nohup python -u one_run_script.py i > one_run.log 2>&1 &
```

Here, i is your perturbation type id.

If in any case you need to stop the script,

In Linux you can use kill command with the PID. First kill the one_run_script then the others (using a script or manually).

Don't filter by 'python' and kill all because it can stop other important python processes.

In Windows, use

```
taskkill /F /FI "WINDOWTITLE eq ollama_cloud_async.py"
```

to kill all the subprocesses the script created.

Before running: In both files, you will see 'df = df.iloc[:2]'. *Comment this.*